

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Monal Raj. B	Reg.No.:	192511176
Department:	CSE	Date:	12.08.2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors. —
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

SIMATS ENGINEERING

Department of Computer Science & Engineering ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Monal Raj.B	Reg.No.:	192511176
Department:	B.E-CSE	Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Given:

+45 and -23

Step 1: Convert to Binary (8-bit)

+45:

45 = 00101101

+23:

23 = 00010111

Step 2: Convert +23 to -23 (2's complement)

1's complement:

00010111 → 11101000

Add 1:

$$11101000 + 1 = 11101001$$

Therefore:

$$-23 = 11101001$$

A. ADDITION: $45 + (-23)$

$$00101101 \quad (+45)$$

$$+ 11101001 \quad (-23)$$

$$00010110 \quad (\text{ignore carry out})$$

Result:

$$00010110 = +22$$

Verification:

$$45 - 23 = 22 \text{ (correct)}$$

Overflow Analysis

Rule:

Overflow occurs if:

1. Two positive numbers give negative result
2. Two negative numbers give positive result

Here:

Operands are of opposite signs $\rightarrow$ No overflow

Hardware Detection:

Overflow = Carry into MSB XOR Carry out of MSB

B. SUBTRACTION: $45 - (-23) = 45 + 23$

$$00101101$$

$$+ 00010111$$

$$01000100$$

Result:

$$01000100 = 68$$

Verification:

$$45 + 23 = 68 \text{ (correct)}$$

Overflow Check

Both operands are positive and result is positive

Therefore:

No overflow occurs

FINAL RESULT

$$45 + (-23) = 22$$

$$45 - (-23) = 68$$

No overflow occurs in both operations

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Introduction

In digital systems, addition is a fundamental operation. A Ripple Carry Adder (RCA) is simple but slow because each bit must wait for the carry from the previous bit. To overcome this delay, a Carry Look-Ahead Adder (CLA) is used.

The CLA improves speed by computing carry signals in advance using Generate and Propagate functions.

Generate and Propagate Signals

For each bit position i :

$$G_i = A_i \cdot B_i \quad (\text{Generate})$$

$$P_i = A_i \oplus B_i \quad (\text{Propagate})$$

Where:

A_i and B_i are input bits

Meaning:

$G_i = 1 \rightarrow$ Carry is generated at that bit

$P_i = 1 \rightarrow$ Carry is propagated to next bit

Carry Computation (4-bit CLA)

Let C_0 be the input carry.

$$C_1 = G_0 + (P_0 \cdot C_0)$$

$$C_2 = G_1 + (P_1 \cdot G_0) + (P_1 \cdot P_0 \cdot C_0)$$

$$C_3 = G_2 + (P_2 \cdot G_1) + (P_2 \cdot P_1 \cdot G_0) + (P_2 \cdot P_1 \cdot P_0 \cdot C_0)$$

$$C_4 = G_3 + (P_3 \cdot G_2) + (P_3 \cdot P_2 \cdot G_1) + (P_3 \cdot P_2 \cdot P_1 \cdot G_0) + (P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0)$$

Thus, all carry values are computed directly without waiting.

Sum Computation

Sum at each bit:

$$S_i = P_i \oplus C_i$$

Where C_i is the carry input to that stage.

Example (4-bit Addition)

Let:

$$A = 1011$$

$$B = 0110$$

Step 1: Compute G and P

Bit 0:

$$G_0 = 1 \cdot 0 = 0$$

$$P_0 = 1 \oplus 0 = 1$$

Bit 1:

$$G_1 = 1 \cdot 1 = 1$$

$$P_1 = 1 \oplus 1 = 0$$

Bit 2:

$$G_2 = 0 \cdot 1 = 0$$

$$P_2 = 0 \oplus 1 = 1$$

Bit 3:

$$G_3 = 1 \cdot 0 = 0$$

$$P_3 = 1 \oplus 0 = 1$$

Step 2: Compute Carries (Assume $C_0 = 0$)

$$C_1 = G_0 + P_0 C_0 = 0 + (1 \cdot 0) = 0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$= 1 + (0 \cdot 0) + (0 \cdot 1 \cdot 0) = 1$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$= 0 + (1 \cdot 1) + (1 \cdot 0 \cdot 0) + (1 \cdot 0 \cdot 1 \cdot 0)$$

$$= 1$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

$$= 0 + (1 \cdot 0) + (1 \cdot 1 \cdot 1) + (1 \cdot 1 \cdot 0 \cdot 0) + (1 \cdot 1 \cdot 0 \cdot 1 \cdot 0)$$

$$= 1$$

Step 3: Compute Sum

$$S_0 = P_0 \oplus C_0 = 1 \oplus 0 = 1$$

$$S_1 = P_1 \oplus C_1 = 0 \oplus 0 = 0$$

$$S_2 = P_2 \oplus C_2 = 1 \oplus 1 = 0$$

$$S_3 = P_3 \oplus C_3 = 1 \oplus 1 = 0$$

$$\text{Final Sum} = 0001$$

$$\text{Carry out} = C_4 = 1$$

$$\text{Result} = 10001 \text{ (17 in decimal)}$$

Time Delay Analysis

Ripple Carry Adder:

Each carry waits for previous carry

Total delay = $n \times \text{gate delay}$

Time complexity = $O(n)$

Carry Look-Ahead Adder:

Carries computed using logic equations in parallel

Delay depends on gate levels only

Time complexity $\approx O(\log n)$

Why CLA is Faster

1. Carry signals are generated in parallel
2. No sequential propagation of carry
3. Reduces waiting time between stages
4. Suitable for high-speed processors

Conclusion

Carry Look-Ahead Adder significantly improves addition speed compared to Ripple Carry Adder by eliminating carry propagation delay. This makes it widely used in modern high-performance processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Introduction

Booth's Algorithm is used for multiplication of signed binary numbers in 2's complement form. It reduces the number of addition and subtraction operations, especially when there are consecutive 1s in the multiplier.

Step 1: Binary Representation (4-bit)

$+6 = 0110$

To get -6 :

1's complement: $0110 \rightarrow 1001$

Add 1: $1001 + 1 = 1010$

Therefore:

$$-6 = 1010$$

$$+5 = 0101$$

Registers Used

$$A \text{ (Accumulator)} = 0000$$

$$Q \text{ (Multiplier)} = 0101$$

$$Q-1 \text{ (Booth bit)} = 0$$

$$M \text{ (Multiplicand)} = 1010$$

Number of bits = 4 (so 4 cycles)

Booth's Rules

$$Q_0 Q_{-1} = 00 \rightarrow \text{No operation}$$

$$Q_0 Q_{-1} = 01 \rightarrow A = A + M$$

$$Q_0 Q_{-1} = 10 \rightarrow A = A - M$$

$$Q_0 Q_{-1} = 11 \rightarrow \text{No operation}$$

After each operation $\rightarrow$ Arithmetic Right Shift (A, Q, Q-1)

Initial Values:

$$A = 0000$$

$$Q = 0101$$

$$Q-1 = 0$$

$$M = 1010$$

Cycle 1

$$Q_0 Q_{-1} = 10 \rightarrow A = A - M$$

$$A = 0000 - 1010$$

Take 2's complement of M:

$$1010 \rightarrow 0110$$

$$A = 0000 + 0110 = 0110$$

Now perform Arithmetic Right Shift:

Before shift:

$$A = 0110$$

$$Q = 0101$$

$$Q-1 = 0$$

After shift:

$$A = 0011$$

$$Q = 0010$$

$$Q-1 = 1$$

Cycle 2

$$Q_0 Q-1 = 0 \ 1 \rightarrow A = A + M$$

$$A = 0011 + 1010 = 1101$$

Arithmetic Right Shift:

Before shift:

$$A = 1101$$

$$Q = 0010$$

$$Q-1 = 1$$

After shift:

$$A = 1110$$

$$Q = 1001$$

$$Q-1 = 0$$

Cycle 3

$$Q_0 Q-1 = 1 \ 0 \rightarrow A = A - M$$

$$A = 1110 - 1010$$

2's complement of $M = 0110$

$$A = 1110 + 0110 = 1\ 0100$$

Ignore overflow $\rightarrow A = 0100$

Arithmetic Right Shift:

Before shift:

$$A = 0100$$

$$Q = 1001$$

$$Q-1 = 0$$

After shift:

$$A = 0010$$

$$Q = 0100$$

$$Q-1 = 1$$

Cycle 4

$$Q_0\ Q-1 = 0\ 1 \rightarrow A = A + M$$

$$A = 0010 + 1010 = 1100$$

Arithmetic Right Shift:

Before shift:

$$A = 1100$$

$$Q = 0100$$

$$Q-1 = 1$$

After shift:

$$A = 1110$$

$$Q = 0010$$

$$Q-1 = 0$$

Final Result

Combine A and Q:

A = 1110

Q = 0010

Result = 11100010

Convert to Decimal

11100010 is negative (MSB = 1)

Find magnitude:

1's complement $\rightarrow$ 00011101

Add 1 $\rightarrow$ 00011110 = 30

Therefore:

Result = -30

Verification

$-6 \times 5 = -30$ (correct)

Advantages of Booth's Algorithm

1. Handles signed numbers directly in 2's complement form
2. Reduces number of addition/subtraction operations
3. Efficient when multiplier contains consecutive 1s
4. Faster than conventional multiplication methods

Conclusion

Booth's Algorithm provides an efficient method for signed binary multiplication by minimizing operations and using arithmetic shifts, making it suitable for modern processors.

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Introduction

Binary division in digital systems can be performed using two main methods:

1. Restoring Division

2. Non-Restoring Division

Both methods use shift and subtract operations, but differ in how negative remainders are handled.

Given

Dividend = 13 = 1101

Divisor = 3 = 0011

Number of bits = 4

Registers used:

A (Remainder) = 0000

Q (Dividend/Quotient) = 1101

M (Divisor) = 0011

A. RESTORING DIVISION

Algorithm Steps:

1. Shift left (A, Q)
2. $A = A - M$
3. If $A < 0 \rightarrow$ restore ($A = A + M$) and set $Q_0 = 0$
4. If $A \geq 0 \rightarrow$ set $Q_0 = 1$
5. Repeat for n bits

Initial Values

A = 0000

Q = 1101

M = 0011

Cycle 1

Shift left (A,Q):

A = 0001

Q = 1010

$A = A - M$:

$$0001 - 0011 = 1110 \text{ (negative)}$$

Restore:

$$A = 1110 + 0011 = 0001$$

$$\text{Set } Q_0 = 0$$

$$Q = 1010$$

Cycle 2

Shift left:

$$A = 0011$$

$$Q = 0100$$

$$A = A - M:$$

$$0011 - 0011 = 0000 \text{ (non-negative)}$$

$$\text{Set } Q_0 = 1$$

$$Q = 0101$$

Cycle 3

Shift left:

$$A = 0000$$

$$Q = 1010$$

$$A = A - M:$$

$$0000 - 0011 = 1101 \text{ (negative)}$$

Restore:

$$A = 1101 + 0011 = 0000$$

$$\text{Set } Q_0 = 0$$

$$Q = 1010$$

Cycle 4

Shift left:

$$A = 0001$$

$$Q = 0100$$

$$A = A - M:$$

$$0001 - 0011 = 1110 \text{ (negative)}$$

Restore:

$$A = 1110 + 0011 = 0001$$

$$\text{Set } Q_0 = 0$$

$$Q = 0100$$

Final Result (Restoring)

$$\text{Quotient (Q)} = 0100 = 4$$

$$\text{Remainder (A)} = 0001 = 1$$

B. NON-RESTORING DIVISION

Key Idea:

- No restoring step is performed
- Next operation depends on sign of remainder

Rules:

$$\text{If } A \geq 0 \rightarrow A = A - M$$

$$\text{If } A < 0 \rightarrow A = A + M$$

After operation $\rightarrow$ shift left (A, Q)

Initial Values

$$A = 0000$$

$$Q = 1101$$

$$M = 0011$$

Cycle 1

$$A \geq 0 \rightarrow A = A - M$$

$$0000 - 0011 = 1101 \text{ (negative)}$$

Shift left:

$$A = 1011$$

$$Q = 1010$$

$$Q_0 = 0$$

Cycle 2

$$A < 0 \rightarrow A = A + M$$

$$1011 + 0011 = 1110 \text{ (still negative)}$$

Shift left:

$$A = 1100$$

$$Q = 0100$$

$$Q_0 = 0$$

Cycle 3

$$A < 0 \rightarrow A = A + M$$

$$1100 + 0011 = 1111 \text{ (negative)}$$

Shift left:

$$A = 1110$$

$$Q = 1000$$

$$Q_0 = 0$$

Cycle 4

$$A < 0 \rightarrow A = A + M$$

$$1110 + 0011 = 0001 \text{ (positive)}$$

Shift left:

$$A = 0011$$

$$Q = 0001$$

$$Q_0 = 1$$

Final Correction

Since A is positive $\rightarrow$ no correction needed

Final Result (Non-Restoring)

Quotient = 0100 = 4

Remainder = 0001 = 1

Comparison of Methods

Restoring Division:

- Includes restore step when remainder is negative
- More operations
- Slower

Non-Restoring Division:

- Eliminates restore step
- Fewer operations
- Faster execution

Conclusion

Both methods give the same result, but non-restoring division is preferred in modern processors because it reduces computation steps and improves efficiency.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

QUESTION 5: IEEE 754 FLOATING POINT REPRESENTATION (-13.25)

Introduction

IEEE 754 is a standard for representing real (floating-point) numbers in binary form. It defines formats such as single precision (32-bit) and double precision (64-bit), consisting of:

1. Sign bit
2. Exponent field (with bias)
3. Mantissa (fraction)

Step 1: Convert Decimal to Binary

Given number: -13.25

Convert integer part (13):

$$13 \div 2 \rightarrow 1101$$

Convert fractional part (0.25):

$$0.25 \times 2 = 0.5 \rightarrow 0$$

$$0.5 \times 2 = 1.0 \rightarrow 1$$

Therefore:

$$13.25 = 1101.01$$

Step 2: Normalize the Number

$$1101.01 = 1.10101 \times 2^3$$

A. SINGLE PRECISION (32-bit)

Format:

Sign (1 bit) | Exponent (8 bits) | Mantissa (23 bits)

Sign Bit

Number is negative $\rightarrow$ Sign = 1

Exponent Field

$$\text{Bias} = 127$$

$$\text{Actual exponent} = 3$$

$$\text{Stored exponent} = 127 + 3 = 130$$

$$130 \text{ in binary} = 10000010$$

Mantissa (Fraction)

Take fractional part after normalization:

$$1.10101 \rightarrow \text{remove leading 1} \rightarrow 10101$$

Fill remaining bits with zeros to make 23 bits:

$$10101000000000000000000$$

Final 32-bit Representation

$$1 \mid 10000010 \mid 10101000000000000000000$$

B. DOUBLE PRECISION (64-bit)

Format:

Sign (1 bit) | Exponent (11 bits) | Mantissa (52 bits)

Sign Bit

Sign = 1

Exponent Field

Bias = 1023

Actual exponent = 3

Stored exponent = $1023 + 3 = 1026$

1026 in binary = 10000000010

Mantissa

Fraction = 10101 followed by zeros up to 52 bits

Final 64-bit Representation

$1 \mid 100000000010 \mid 1010100$

FLOATING POINT ADDITION

Steps involved in adding two floating-point numbers:

1. Align exponents:

- Compare exponents
- Shift mantissa of smaller exponent to the right

2. Add mantissas:

- Perform binary addition

3. Normalize result:

- Adjust mantissa to form 1.x format
- Update exponent accordingly

4. Rounding:

- Round result to fit available bits (IEEE uses round-to-nearest)

Precision and Rounding Issues

1. Limited precision:

Only finite bits available for mantissa

2. Rounding errors:

Values are approximated

3. Loss of significance:

When adding numbers with large exponent difference

4. Some decimals cannot be exactly represented:

Example: 0.1 in decimal has no exact binary equivalent

Conclusion

IEEE 754 provides a standardized way to represent floating-point numbers in computers. While it ensures consistency across systems, it introduces rounding errors and precision limitations due to finite storage.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					

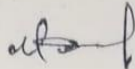
Q5					
----	--	--	--	--	--

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Code of Conduct:

I Honal Raji B (192511176) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1	4	3	3	2	2
Q2	3	4	4	2	2
Q3	4	3	3	3	3
Q4	3	4	4	4	2
Q5	4	4	3	3	2

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation